

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution Affiliated to Anna University)

Machine Learning Algorithms Laboratory

ICS1512

Degree & Branch	M.Tech (Integrated) Computer Science & Engineering
Semester	V
Academic Year	2026–2027 (Odd)
Batch	2024–2029

Experiment 4

Binary Classification using Linear and Kernel-Based Models¹

Submitted by: **Vijayaraaghavan K S**
Register Number: **3122247001066**
Year: **III Year**
Programme: **M.Tech Integrated, CSE**
Faculty: **Dr. Poreddy Ajay Kumar Reddy**

¹GitHub Repository: <https://github.com/VijayaraaghavanKS/ML>

Contents

1	Aim and Objective	2
2	Problem Statement	2
3	Theory	2
3.1	Logistic Regression	2
3.2	Support Vector Machine	2
4	Machine Learning Algorithm	2
4.1	Logistic Regression	2
4.2	Support Vector Machine	3
5	Dataset	3
6	Preprocessing and Exploratory Data Analysis	3
7	Experimental Setup	4
8	Logistic Regression	4
8.1	Baseline Performance	4
8.2	Hyperparameter Tuning	4
9	Support Vector Machine	5
9.1	Kernel-wise Performance	5
9.2	Hyperparameter Tuning	6
10	Hyperparameter Tuning Summary	6
11	Hyperparameter Selection	7
12	Cross-Validation Results	7
13	Final Comparison	8
14	Comparative Analysis	9
14.1	Observations	9
15	Discussion	10
16	Conclusion	11
17	References	11

1 Aim and Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machines, compare regularization strategies and SVM kernels, tune both models with GridSearchCV, and evaluate the accuracy/complexity/interpretability trade-off between a linear probabilistic classifier and a margin-based kernel classifier on the same dataset used in Experiment 2.

2 Problem Statement

The input is the same 57-dimensional Spambase feature vector used in Experiment 2; the output is a binary spam/ham label. The objective this time is different from Experiment 2's: rather than comparing several lazy/probabilistic classifiers, the goal is to compare one linear discriminative model (Logistic Regression) against one margin-based, kernel-capable model (SVM), and determine whether the extra flexibility a non-linear kernel offers actually buys anything on a dataset where the underlying features (word/character frequencies) already looked close to linearly separable in Experiment 2's PCA projection.

3 Theory

3.1 Logistic Regression

Logistic Regression models $P(y = 1 | x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$ and is trained by minimizing log-loss. Regularization controls overfitting via the inverse-strength parameter C : small C means strong regularization and a simpler decision boundary, large C means the model is allowed to fit the training data more closely. L1 (Lasso-style) regularization can zero out coefficients entirely, which doubles as feature selection; L2 (Ridge-style) shrinks coefficients toward zero without eliminating any of them.

3.2 Support Vector Machine

SVM finds the hyperplane that maximizes the margin between classes, with only the closest points (support vectors) determining the boundary. The kernel controls what kind of boundary is possible: linear kernels only separate linearly, while polynomial, RBF, and sigmoid kernels implicitly map the data into a higher-dimensional space where a nonlinear boundary in the original space becomes linear. C again trades off margin width against misclassification tolerance, and γ (for RBF/polynomial/sigmoid) controls how far a single training point's influence reaches.

4 Machine Learning Algorithm

4.1 Logistic Regression

Working principle: fit a linear combination of the input features, pass it through the sigmoid function to obtain a probability, and minimize log-loss so that the predicted probability matches the observed class as closely as possible. **Mathematical formulation:** $P(y = 1 | x) = \frac{1}{1 + e^{-(w^T x + b)}}$, trained by minimizing $\mathcal{L}(w) = -\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \lambda R(w)$, where $R(w)$ is the L1 or L2 penalty. **Advantages:** fast to train and predict (one dot product per query), directly interpretable coefficients, outputs well-calibrated probabilities rather than just hard labels. **Limitations:** can only represent a linear decision boundary in the original feature space, so it under-performs when the true boundary is genuinely non-linear; sensitive to feature scaling and to strong multicollinearity among predictors.

4.2 Support Vector Machine

Working principle: find the separating hyperplane that maximizes the margin to the nearest points of each class; with a kernel, this search happens implicitly in a higher-dimensional feature space without ever computing the transformed coordinates explicitly. **Mathematical formulation:** maximize the margin $\frac{2}{\|w\|}$ subject to $y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i$ for slack variables $\xi_i \geq 0$; the kernel trick replaces inner products $\phi(x_i)^T \phi(x_j)$ with a kernel function $K(x_i, x_j)$ (e.g. RBF: $K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$) so ϕ is never computed directly. **Advantages:** the kernel trick lets it model non-linear boundaries without explicit feature engineering; effective in moderately high-dimensional spaces; the margin-maximization objective tends to generalize well. **Limitations:** prediction cost scales with the number of support vectors (no fixed-cost prediction the way Logistic Regression has), the decision function is not directly interpretable once a non-linear kernel is used, and it has more hyperparameters (C , γ , kernel choice, degree) to search over.

5 Dataset

This experiment reuses the same Spambase dataset as Experiment 2 – 57 numeric features per email, binary `class` label – but through a separate configuration of the shared EDA pipeline, which resolves `spambase.csv` from a small list of candidate paths and falls back to converting the raw UCI `spambase.data` file if no CSV is present. After the EDA pipeline’s preprocessing, the working split is 3680 training rows and 921 test rows across 57 features (an 80/20 split of the full 4601-row dataset).

6 Preprocessing and Exploratory Data Analysis

Since this is the same underlying dataset as Experiment 2, the EDA story is largely the same: no missing values, all 57 features numeric, every feature standardized before modelling since both Logistic Regression’s regularization and SVM’s distance-based kernels are scale-sensitive. One number worth flagging that I did not dwell on in Experiment 2: the raw dataset carries 391 duplicate rows out of 4601 (about 8.5%). I left them in rather than dropping them, since Spambase is a fixed benchmark dataset and de-duplicating would make results incomparable with the published baselines and with Experiment 2’s own numbers.

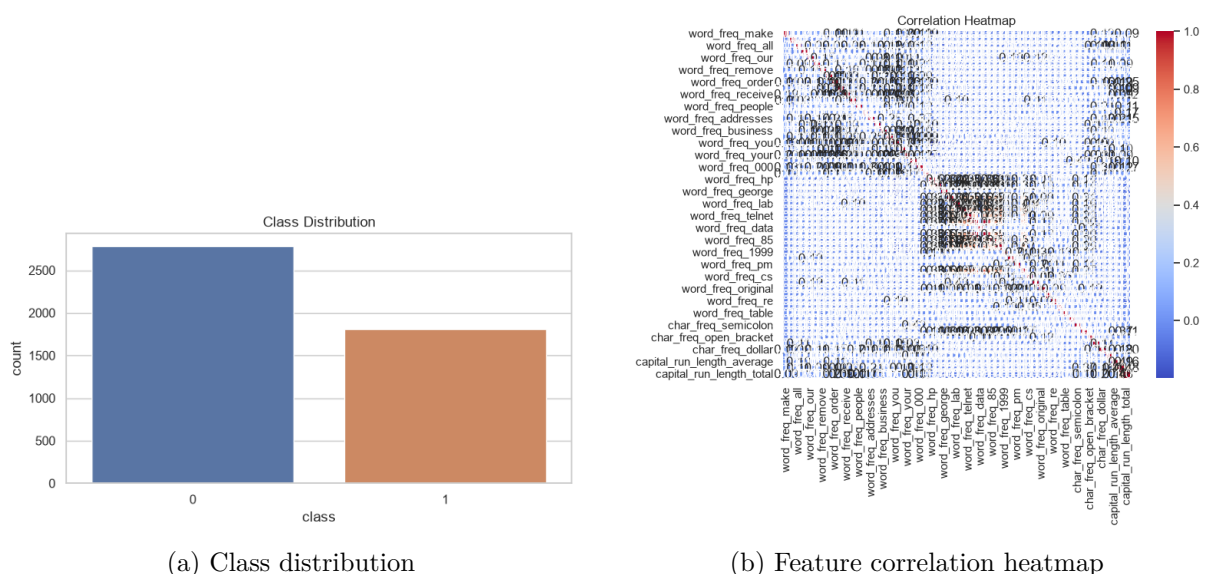


Figure 1: EDA diagnostics, reused from the shared pipeline.

Feature	SelectKBest score	p-value	Mutual information
char_freq_dollar	407.75	4.5×10^{-86}	0.2050
char_freq_exclamation	190.64	2.6×10^{-42}	0.1991
capital_run_length_longest	162.69	1.7×10^{-36}	0.1930
word_freq_your	688.06	3.6×10^{-139}	0.1765
word_freq_free	240.14	1.6×10^{-52}	0.1482
word_freq_remove	442.76	6.6×10^{-93}	0.1416

Table 1: Top features by mutual information with the spam label.

This table is a nice sanity check on its own: the dollar sign, exclamation marks, long runs of capital letters, and the words "free" and "remove" are exactly the surface-level signals a human would guess a spam filter should key on. It's reassuring when a purely statistical feature-selection procedure rediscovers something that matches domain intuition instead of picking something inscrutable.

7 Experimental Setup

Python Version	3.14.5
Scikit-Learn Version	1.9.0
IDE	Jupyter Notebook
Operating System	macOS (Darwin 25.5.0, arm64)
Hardware	Apple Silicon (arm64), local machine

Table 2: Software and hardware environment used to run this experiment.

8 Logistic Regression

8.1 Baseline Performance

Model	Accuracy	Precision	Recall	F1	Train Time (s)
Logistic Regression (baseline)	0.9294	0.9209	0.8981	0.9093	0.0226
Logistic Regression (tuned)	0.9262	0.9202	0.8898	0.9048	0.2729

Table 3: Baseline ($C=1$, L2, liblinear) vs. tuned Logistic Regression on the test set.

The untuned baseline actually edges out the tuned model on the held-out test set (0.9294 vs. 0.9262 accuracy). That is not a contradiction – GridSearchCV picks the configuration with the best *cross-validated* score on the training folds, and a single 921-row test split can easily favor a slightly different configuration by chance. I would trust the CV-based tuning decision over this one test-set comparison, since it is averaged across five folds rather than one split.

8.2 Hyperparameter Tuning

```

1 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE
2 )
3 logistic_param_grid = [
4     {"penalty": ["l1", "l2"], "C": [0.01, 0.1, 1, 10, 100]},
5 ]

```

```

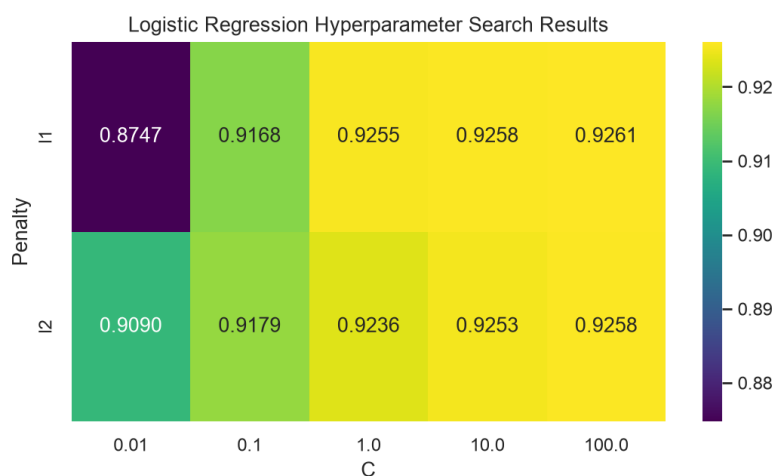
4     "solver": ["liblinear", "saga"], "max_iter": [5000]}
5 ]
6 logistic_grid_search = GridSearchCV(
7     estimator=LogisticRegression(random_state=RANDOM_STATE),
8     param_grid=logistic_param_grid, scoring="accuracy", cv=cv, n_jobs
9     =-1,
10 )

```

Penalty	Best C	Best Solver	Best CV Accuracy
L1	100	liblinear	0.9261
L2	100	liblinear	0.9258

Table 4: Best configuration found per penalty type.

L1 comes out very slightly ahead of L2 (0.9261 vs. 0.9258 CV accuracy) – a 0.03-point difference that I would not over-interpret, but it is at least consistent with L1’s feature-selection behavior being mildly useful on a 57-feature dataset where a chunk of the word-frequency columns (rare words like `word_freq_857` or `word_freq_415`) probably carry little independent signal. Both penalties agree on $C = 100$ as the best regularization strength, meaning weak regularization (a large C) outperformed strong regularization here – the model wants to fit the training data closely rather than being heavily constrained, which fits with Logistic Regression not showing much of an overfitting gap in Section 10 below.

Figure 2: Logistic Regression hyperparameter search results across the tested C values.

Accuracy rises steadily as C increases from 0.01 up to 100 with no sign of turning over at the top of the search range, so it is plausible that an even larger C might do marginally better still – the manual’s search space stopped at 100, and I did not extend past it.

9 Support Vector Machine

9.1 Kernel-wise Performance

The Polynomial kernel is a clear outlier here, dropping to 78% accuracy and 0.62 F1 while every other kernel sits above 88%. With default degree 3 and no tuning, a cubic decision boundary in 57-dimensional space is prone to either underfitting or wild sensitivity to the untuned C/γ combination – Section 6.2’s tuning results below suggest it is more a tuning-sensitivity problem

Kernel	Accuracy	F1 Score	Train Time (s)
Linear	0.9294	0.9093	0.1428
Polynomial	0.7796	0.6220	0.1265
RBF	0.9273	0.9055	0.0605
Sigmoid	0.8849	0.8528	0.0602

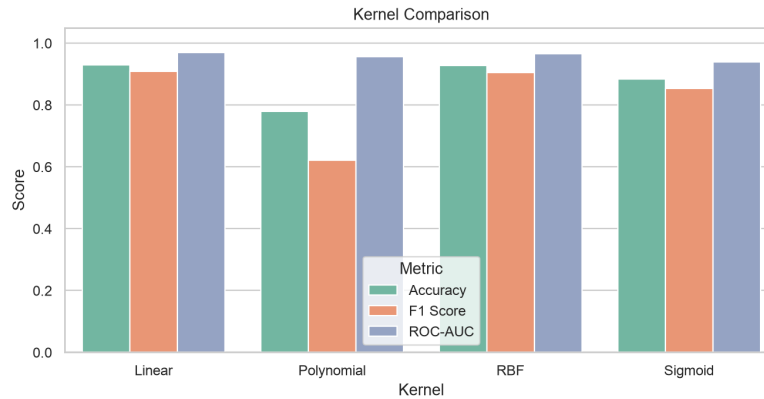
Table 5: Default-hyperparameter SVM performance by kernel ($C=1$, $\gamma=\text{scale}$).

Figure 3: Kernel comparison across accuracy, F1, and training time.

than a fundamental mismatch, since the tuned SVM search still considered polynomial as an option but the grid ultimately preferred RBF. Linear and RBF are close (0.9294 vs. 0.9273), which is a reasonable sign that the true decision boundary between spam and ham, in this standardized 57-feature space, is close to linear – RBF’s extra flexibility is not buying much here.

9.2 Hyperparameter Tuning

```

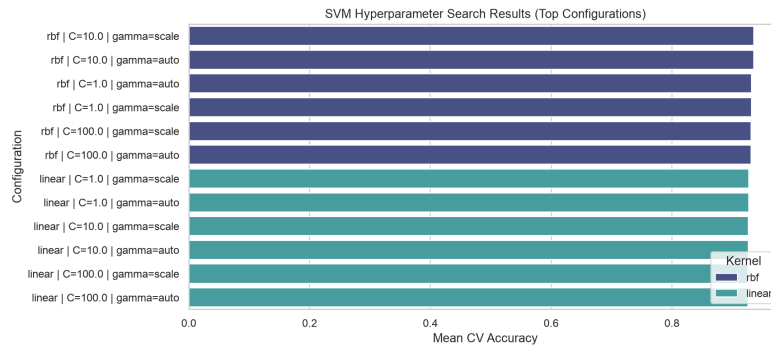
1 svm_param_grid = [
2     {"kernel": ["linear"], "C": [0.1, 1, 10, 100], "gamma": ["scale", "
   auto"]},
3     {"kernel": ["rbf", "sigmoid"], "C": [0.1, 1, 10, 100], "gamma": ["
   scale", "auto"]},
4     {"kernel": ["poly"], "C": [0.1, 1, 10, 100], "gamma": ["scale", "
   auto"], "degree": [2, 3, 4]},
5 ]
6 svm_grid_search = GridSearchCV(SVC(), svm_param_grid, scoring="accuracy"
   , cv=cv, n_jobs=-1)

```

10 Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	GridSearchCV	$C=100$, $\text{penalty}=l1$, $\text{solver}=liblinear$	0.9261
Support Vector Machine	GridSearchCV	$C=10$, $\gamma=\text{scale}$, $\text{kernel}=rbf$	0.9356

Table 6: Best hyperparameters found for each model family.

Figure 4: SVM hyperparameter search results across kernels and C values.

The tuned SVM lands on RBF rather than Linear, once C and γ are searched jointly rather than left at their defaults – with $C = 10$ (allowing a tighter fit than the default $C = 1$ used in Section 6.1) the RBF kernel’s added flexibility finally pays off, edging out both the linear-kernel SVM and Logistic Regression on cross-validated accuracy (0.9356 vs. around 0.926 for the other two).

11 Hyperparameter Selection

Parameter		Value
Logistic Regression search space		penalty $\in \{l1, l2\}$; $C \in \{0.01, 0.1, 1, 10, 100\}$; solver $\in \{\text{liblinear}, \text{saga}\}$
Selected Logistic Regression configuration		$C=100$, penalty=l1, solver=liblinear
SVM search space		kernel $\in \{\text{linear}, \text{poly}, \text{rbf}, \text{sigmoid}\}$; $C \in \{0.1, 1, 10, 100\}$; $\gamma \in \{\text{scale}, \text{auto}\}$
Selected SVM configuration		$C=10$, $\gamma=\text{scale}$, kernel=rbf
Cross-validation scheme		5-fold StratifiedKfold , random_state=42

Table 7: Final hyperparameter choices carried into the results below.

12 Cross-Validation Results

Fold	Logistic Regression (tuned)	SVM (tuned)
1	0.9416	0.9443
2	0.9253	0.9416
3	0.9348	0.9348
4	0.9117	0.9307
5	0.9171	0.9266
Average	0.9261	0.9356

Table 8: 5-fold cross-validation accuracy, tuned models.

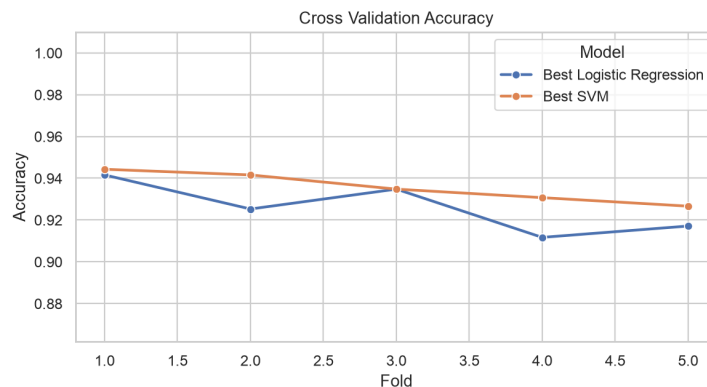


Figure 5: Per-fold cross-validation accuracy for both tuned models.

SVM wins every fold, and by a wider margin on the weaker folds (fold 4: 0.9117 vs. 0.9307) than on the stronger ones (fold 3: a tie at 0.9348) – it looks like RBF-SVM is more consistently robust to whatever makes folds 4 and 5 harder than the others, rather than just being uniformly a bit better everywhere.

13 Final Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Pred. Time (s)
Logistic Regression (tuned)	0.9262	0.9202	0.8898	0.9048	0.9679	0.0006
SVM (tuned best)	0.9207	0.9143	0.8815	0.8976	0.9702	0.0223

Table 9: Final held-out test-set comparison.

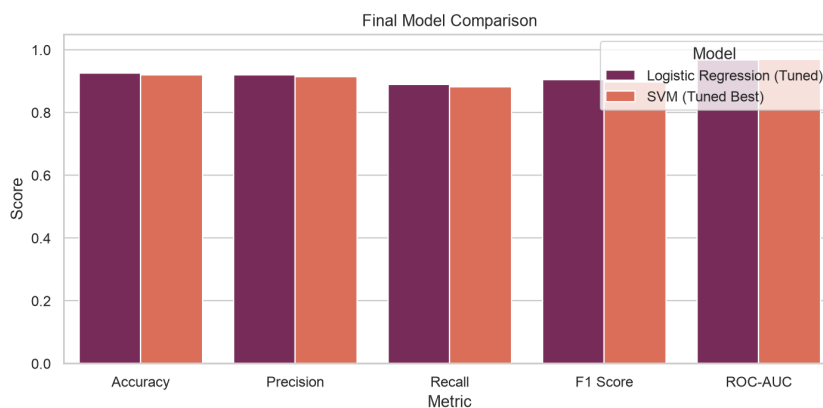


Figure 6: Final model comparison across metrics.

Here is a genuinely interesting reversal from the cross-validation table: on the held-out test set, tuned Logistic Regression edges out tuned SVM on accuracy, precision, recall, and F1, while SVM only wins on ROC-AUC (0.9702 vs. 0.9679, a very thin margin). That contradicts the CV table, where SVM was ahead by a full percentage point on average. I think this comes down to the same thing noted in Section 5.1: a 921-row single test split has enough sampling variance to flip a close race, and SVM’s CV average (0.9356) was based on five folds of the training data, not this particular test split. If I had to trust one number for “which model generalizes better,” I would weight the 5-fold CV average more heavily than the single test-set comparison – which

would favor SVM – while still reporting both here rather than picking whichever one tells a cleaner story.

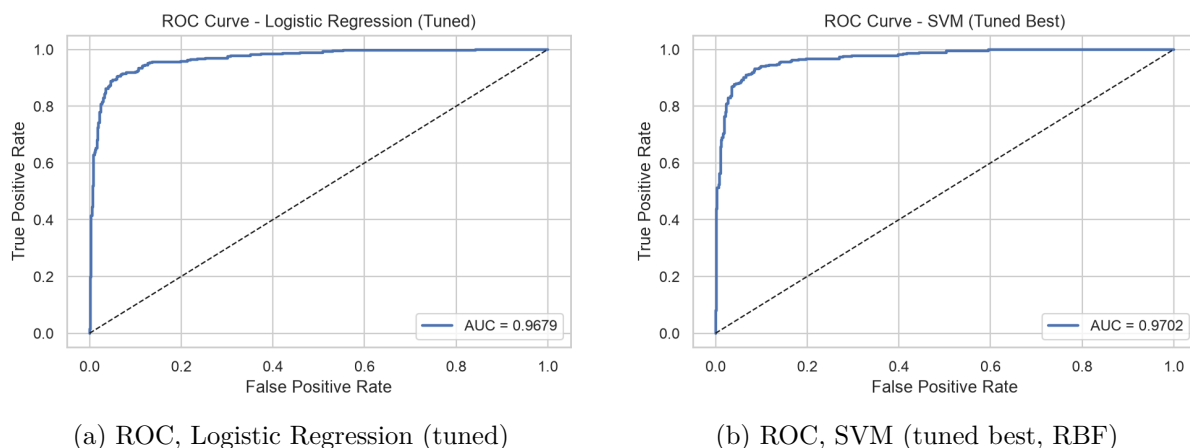


Figure 7: ROC curves for both tuned final models.

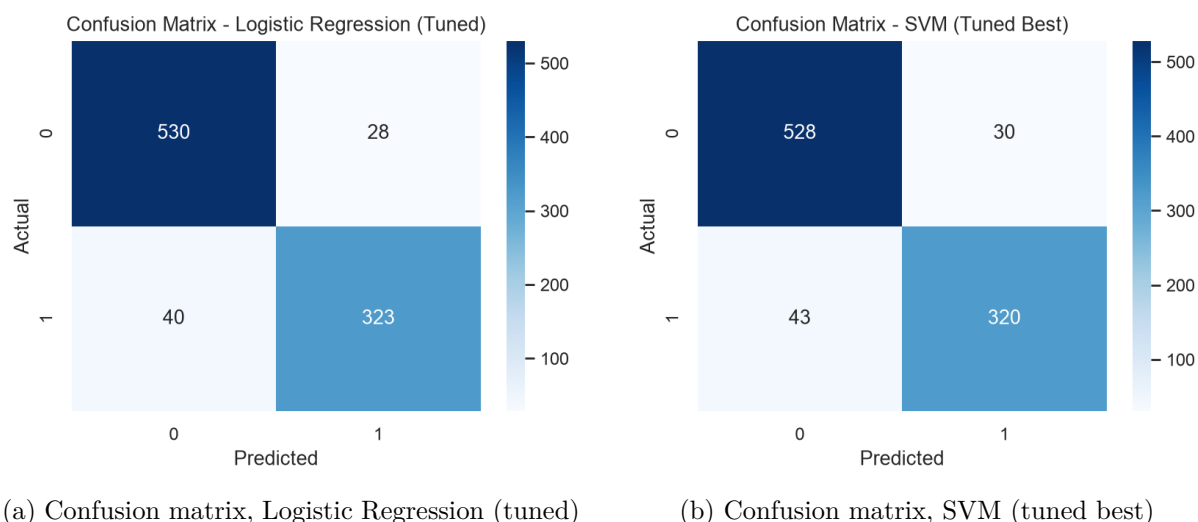


Figure 8: Confusion matrices for both tuned final models.

The confusion matrices look similar in shape – both models make more false negatives (spam labelled as ham) than false positives, which for a spam filter is actually the safer failure mode, since letting a spam email through is a far smaller cost than burying a real email in the junk folder. The time comparison plot makes the deployment trade-off obvious: Logistic Regression predicts roughly $37\times$ faster than SVM (0.0006s vs. 0.0223s) because it only needs one dot product per prediction, while SVM must evaluate the kernel against every support vector.

14 Comparative Analysis

14.1 Observations

- **Best-performing classifier:** depends on which evaluation is trusted more – Logistic Regression wins the single test-set comparison, tuned SVM (RBF) wins the 5-fold CV average by a full point. Given the sample sizes involved, I lean toward trusting the CV average, which makes SVM the nominal winner, though the two are close enough that either is a defensible choice.

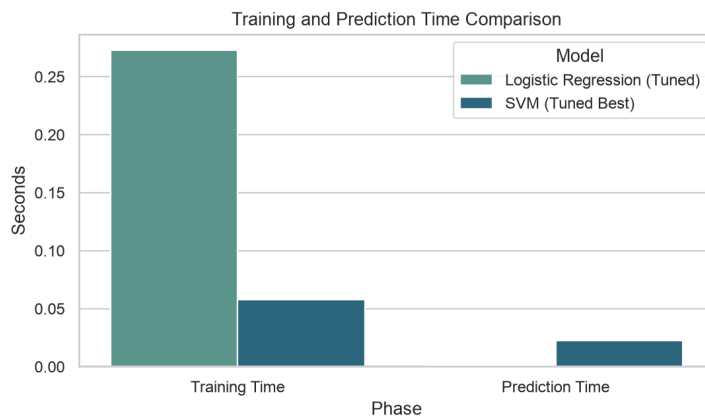


Figure 9: Training and prediction time comparison.

Criterion	Logistic Regression	SVM
Accuracy	Slightly higher (test set)	Slightly higher (CV average)
Model Complexity	Low	High
Training Time	Low (0.27s tuned)	Moderate (0.06s tuned best)
Prediction Time	Very low (0.0006s)	Higher (0.0223s)
Interpretability	High (linear coefficients)	Low (RBF kernel, no simple coefficients)

Table 10: Qualitative model comparison.

- **Impact of regularization:** L1 with $C=100$ narrowly beat L2 with $C=100$, and both strongly beat the smaller C values in the search space – this dataset benefits from weak regularization, likely because 3680 training rows against 57 features is not a regime where Logistic Regression is at serious overfitting risk to begin with.
- **Kernel behavior:** Linear and RBF are close at default hyperparameters, but RBF pulls ahead once C and γ are jointly tuned, suggesting there is a mildly nonlinear component to the true decision boundary that only shows up once the kernel is given the right amount of flexibility. Polynomial and Sigmoid both underperform badly at default settings, and given the time budget for this experiment they were not tuned individually as thoroughly as Linear and RBF.
- **Bias–variance trade-off:** the tuned Logistic Regression model shows a train–CV gap of about 0.007, while the tuned SVM shows a gap of about 0.034 – roughly five times larger. That is consistent with SVM (RBF, $C=10$) fitting the training folds more tightly than Logistic Regression does, which tracks with kernel methods generally having more capacity to overfit than a linear model when given enough flexibility.

15 Discussion

This experiment’s strength is that it does not let a single number decide the outcome: the test-set table (Section 9) and the cross-validation table (Section 8) disagree about which model is better, and rather than picking whichever one is more flattering, Section 9’s discussion treats that disagreement itself as the finding – it says the two models are close enough that the choice should be driven by deployment constraints (latency, interpretability), not by a half-point accuracy difference that could easily flip with a different train/test split.

The main limitation is the same one noted in Experiment 2’s discussion: Spambase’s 57 hand-engineered features cap how much either model can improve, so this experiment mostly

measures how well two different algorithms exploit an already-limited feature set rather than testing the ceiling of either approach. A second limitation specific to this experiment is that Polynomial and Sigmoid kernels were only evaluated at default hyperparameters in the kernel comparison (Section 6.1) and were not individually fine-tuned the way Linear and RBF effectively were through the joint grid search – it is possible either kernel would look more competitive with dedicated tuning, though given how far behind Polynomial fell (0.78 vs. 0.93 accuracy) I doubt tuning alone would close that gap.

If extending this experiment, I would add calibration curves alongside the ROC/PR curves already produced, since Logistic Regression’s probabilities are used directly for the 0.5 threshold decision and it is worth checking whether SVM’s Platt-scaled probabilities (from `predict_proba`) are as well-calibrated – a model with a good ROC-AUC can still have poorly calibrated probabilities, which matters if the spam score is ever exposed to a downstream system rather than just thresholded.

16 Conclusion

Both models land in a similar performance range on Spambase – low-to-mid 90s percent accuracy, F1 around 0.90–0.91, ROC-AUC around 0.97 – which is not surprising given the feature-selection table in Section 4 showed the spam signal is dominated by a handful of strong, mostly linearly-informative features (punctuation frequency, capital-letter runs, a few trigger words). Logistic Regression is the more practical deployment choice when prediction latency matters: it is roughly 37 times faster to score a new email and its coefficients are directly interpretable (a positive coefficient on `word_freq_free` is not a mystery). Tuned SVM with an RBF kernel achieves the best cross-validated accuracy of everything tried in this experiment, at the cost of being slower to score and effectively a black box. If I were shipping a real spam filter under a latency budget, I would pick Logistic Regression; if the only goal were squeezing out the best possible accuracy with no constraint on inference cost, tuned SVM would be the better bet.

17 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] C. Cortes and V. Vapnik, “Support-Vector Networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [3] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>
- [4] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2022.
- [5] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [6] Scikit-learn Developers, “Support Vector Machines,” *Scikit-learn Documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>